

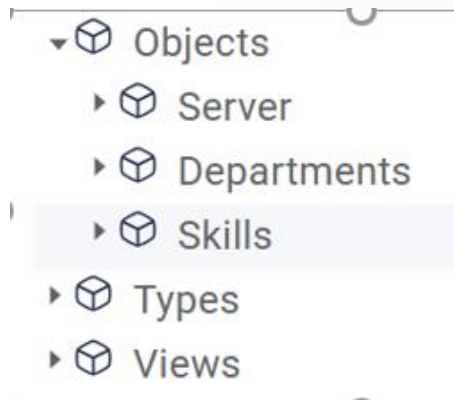
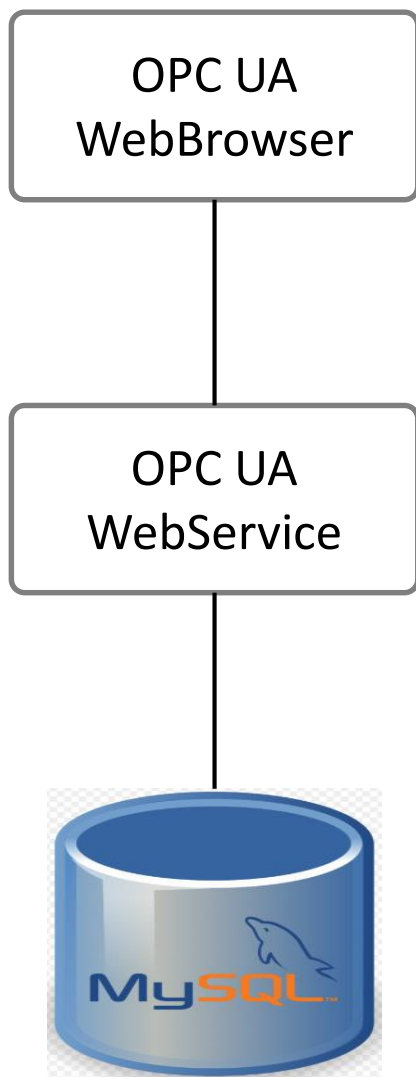


OPC UA WebAPI开发教程

— 开发环境测试

- 1.编译opcua-webservice-spring类库
- 2.编译uaweb servicetest测试程序
- 3.使用opcua-webbrowser测试服务

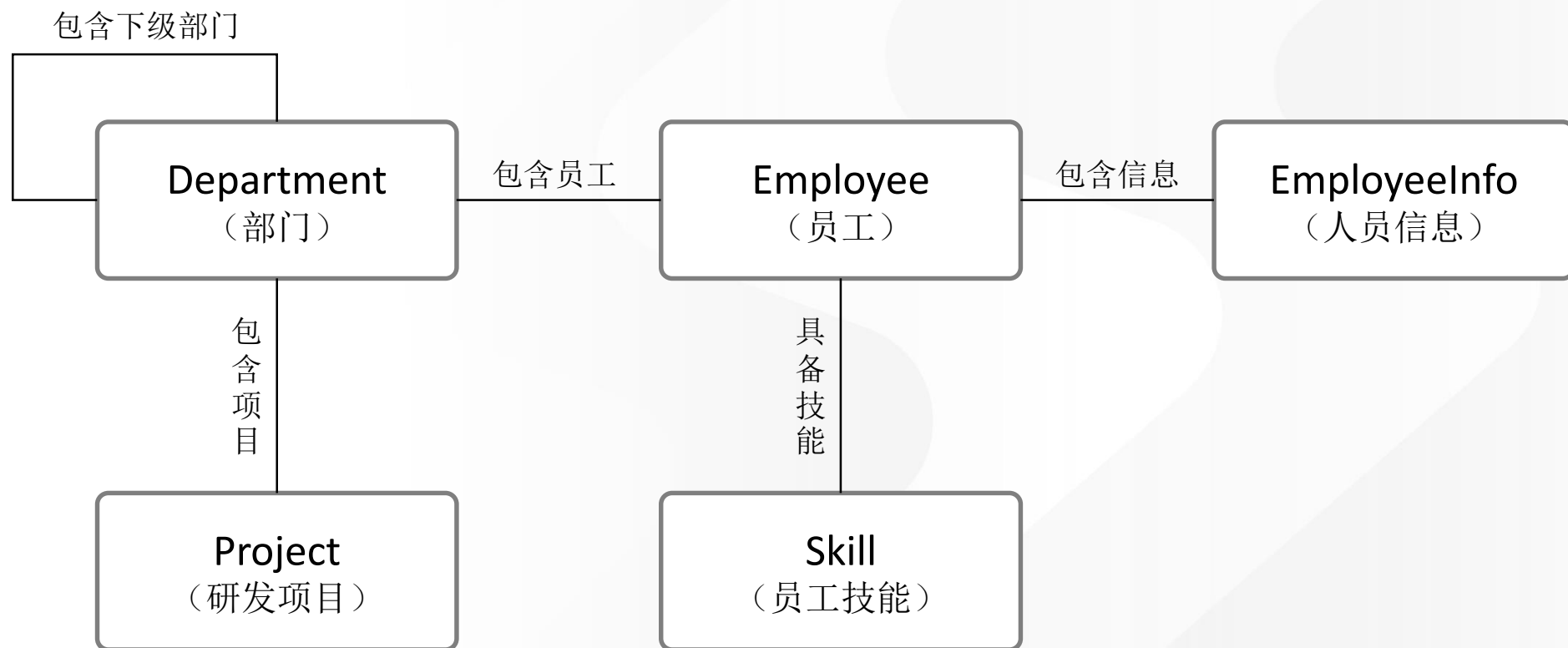
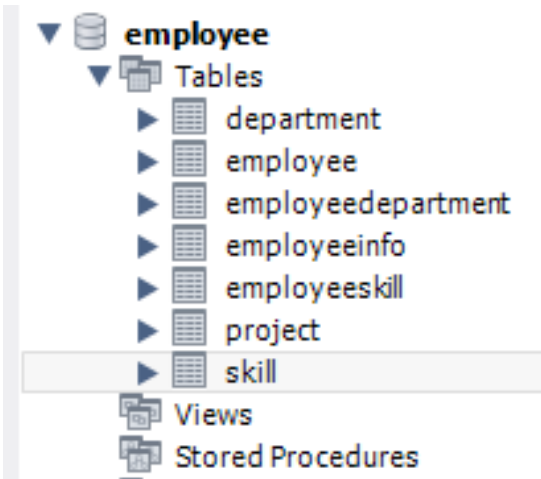
— 课程介绍



Variables		
Name	Data Type	Value
Level	SkillLevel	2(Senior)
Category	SkillCategory	0(Backend developer)
YearOfExperience	Int32	5

Node Information	
nodeId	ns=1;b=eyJvaSI6eyJ0IjoibnM9MTtzPVNraWxsVHlwZSIsImkiOiIzIn19
nodeClass	Object
browseName	3
displayName	BE-DEV-S
description	Senior backend developer

— 课程介绍-数据源



— 课程1

技能列表

Skills

- BE-DEV-J
- BE-DEV-M
- BE-DEV-S
- UI-DEV-J
- UI-DEV-M
- UI-DEV-S
- S-TST-J
- S-TST-M
- S-TST-S

技能基本信息

nodeId	ns=1;b=eyJvaSI6eyJ0ljoibnM9MTtzPVNraWxsVHlwZSIsImkiOi0In19
nodeClass	Object
browseName	4
displayName	UI-DEV-J
description	Junior UI developer

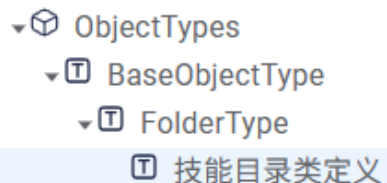
技能数据

Name	Data Type	Value
Level	SkillLevel	0(Junior)
Category	SkillCategory	1(UI developer)
YearOfExperience	Int32	1

ID	SkillName	Description	Level	Category	YearOfExperience
1	BE-DEV-J	Junior backend developer	0	BE_DEV	1
2	BE-DEV-M	Middle level backend developer	1	BE_DEV	3
3	BE-DEV-S	Senior backend developer	2	BE_DEV	5
4	UI-DEV-J	Junior UI developer	0	UI_DEV	1
5	UI-DEV-M	Middle level UI developer	1	UI_DEV	3
6	UI-DEV-S	Senior UI developer	2	UI_DEV	5

— 课程1-SkillDirectoryType

类型声明



对象实例



- SkillDirectoryType负责列出Skill列表
- SkillDirectoryType继承自UaObjectDirectoryType
- SkillDirectoryType模型代码

```
public class SkillDirectoryType extends UaObjectDirectoryType { 3个用法
    public final String RootSkillDirectoryId = "Skills"; 1个用法

    public SkillDirectoryType(EmployeeDataManager nodeManager) {...}

    // Get skill information of employee
    @Override  @dingyan <dy_100000@163.com>
    public CompletableFuture<GetObjectDirectoryChildResponse> getChildren(
        [...]
    )
}
```

- 重载getChildren可返回Skill列表
- 调用SQL语句获取Skill信息
- getChildren代码

```
SELECT ID, SkillName FROM Skill
```

```
// Database query
SkillMapper skillMapper = EmployeeDatabase.getSkillMapper();
List<Skill> skills = skillMapper.getAllSkillInfo();

// Return general information of skill
List<UaChildObjectDescriptor> children = new ArrayList<>();
SkillType skillType = ((EmployeeDataManager)nodeManager).skillType;

for (Skill item : skills)
{
    UaChildObjectDescriptor descriptor = new UaChildObjectDescriptor(
        item.getId().toString(), // SkillId
        new LocalizedText(item.getName()), // SkillName
        skillType); // SkillType

    children.add(descriptor);
}

GetObjectDirectoryChildResponse response = new GetObjectDirectoryChildResponse(children, containsMoreData: false);
return CompletableFuture.completedFuture(response);
```

— 课程1-SkillType

类型声明

- ▼ **BaseObjectType**
 - ▶ FolderType
 - ▶ ServerType
 - ▶ ModellingRuleType
 - ▶ BaseEventType
 - ▶ **技能类定义**

对象描述

nodeId	ns=1;b=eyJvaSi6eyJ0ljoibnM9MTIzPVNfraWxsVHlwZSIsimkiOilyIn19
nodeClass	Object
browseName	2
displayName	BE-DEV-M
description	Middle level backend developer

对象实例

- ▶ BE-DEV-J
- ▶ **BE-DEV-M**
- ▶ BE-DEV-S
- ▶ UI-DEV-J
- ▶ UI-DEV-M

对象数据

Name	Data Type	Value
Level	SkillLevel	1(Middle)
Category	SkillCategory	0(Backend developer)
YearOfExperience	Int32	3

- SkillType负责体现Skill的描述信息及数据
- SkillType继承自UaDataObjectType
- SkillType模型代码

```
public class SkillType extends UaDataObjectType { 4 个用法  @dingyan <dy_100000@163.com>
    private final UaVariable level; 2 个用法
    private final UaVariable category; 2 个用法
    private final UaVariable yearOfExperience; 2 个用法

    public SkillType( 1 个用法  @dingyan <dy_100000@163.com>
        SkillLevelEnumType skillLevelEnumType,
        SkillCategoryEnumType skillCategoryEnumType,
        EmployeeDataManager nodeManager) {...}

    // Return general information such as name, description
    public CompletableFuture<ReadObjectAttributeResponse> getObjectAttribute(ReadObjectAttributeRequest request) {...}

    @Override 0 个用法  @dingyan <dy_100000@163.com>
    public CompletableFuture<ReadVariableValueResponse> getVariableValues(ReadVariableValueRequest request) {...}
}
```


— 课程1-SkillType

- 在构造函数中提供类的名称和描述
- 在构造函数中定义数据结构
- SkillType构造函数代码

```
super(objectType: "SkillType", new LocalizedText("技能类型"), p
setDescription(new LocalizedText("To present the data of employee

// Add start time data field
level = addVariable(
    memberId: "Level",
    mandatory: true,
    new LocalizedText("Level"),
    new LocalizedText("The skill level of employee"),
    skillLevelEnumType,
    writable: false,
    historizing: false,
    valueRank: null,
    variableType: null);
```

```
// Add end time data field
category = addVariable(
    memberId: "Category",
    mandatory: true,
    new LocalizedText("Category"),
    new LocalizedText("The category of skill"),
    skillCategoryEnumType,
    writable: false,
    historizing: false,
    valueRank: null,
    variableType: null);

// Add budget data field
yearOfExperience = addVariable(
    memberId: "YearOfExperience",
    mandatory: true,
    new LocalizedText("YearOfExperience"),
    new LocalizedText("The year of experience required to be qualified"),
    UaDataTypes.Int32,
    writable: false,
    historizing: false,
    valueRank: null,
    variableType: null);
```

- 重载getObjectAttribute返回Skill描述
- 调用SQL语句获取Skill描述信息
- getObjectAttribute代码

```
SELECT ID, SkillName, Description FROM Skill WHERE ID = #{id}
```

```
// Get department id from request
Integer skillId = Integer.parseInt(request.getObjectId().getId());

// Database query
SkillMapper skillMapper = EmployeeDatabase.getSkillMapper();
Skill skill = skillMapper.getSkillInfo(skillId);
if (null == skill) throw new UaRuntimeException(StatusCodes.Bad_NodeIdUnknown);

// Return department general information
ReadObjectAttributeResponse response = new ReadObjectAttributeResponse(
    new LocalizedText(skill.getName()), // Object name
    new LocalizedText(skill.getDescription())); // Object description

return CompletableFuture.completedFuture(response);
```


— 课程1-SkillType

- 重载getVariableValues返回Skill数据
- 调用SQL获取Skill数据

```
SELECT Level,Category,YearOfExperience FROM Skill WHERE ID = #{id}
```

- getVariableValues代码

```
// Get skill id from request
Integer skillId = Integer.parseInt(request.getObjectId().getId());

// Database query
SkillMapper skillMapper = EmployeeDatabase.getSkillMapper();
Skill skill = skillMapper.getSkillData(skillId);
if (null == skill) throw new RuntimeException(StatusCodes.Bad_NodeIdUnknown);

// Return field data
ReadVariableValueResponse response = new ReadVariableValueResponse();

for (UaChildId item: request.getVariableIds())
{
    // Return level if required
    if (Level.browseName().equals(item.getPathId()))
    {
        SkillLevel level = SkillLevel.fromInt(skill.getLevel());
        response.setValue(item, Variant.ofInt32(level.getCode()));
    }

    // Return category if required
    if (Category.browseName().equals(item.getPathId())) {
        SkillCategory category = SkillCategory.fromString(skill.getCategory());
        response.setValue(item, Variant.ofInt32(category.getCode()));
    }
}
```

```
// Return yearOfExperience if required
if (YearOfExperience.browseName().equals(item.getPathId())) {
    response.setValue(item, Variant.ofInt32(skill.getYearOfExperience()));
}

return CompletableFuture.completedFuture(response);
```

— 课程1-SkillCategory/SkillLevel枚举

- 定义SkillCategory枚举

```
public enum SkillLevel { 9 个用法  @dingyan <dy_100000@163.com>
    JUNIOR( code: 0, description: "Junior"), 0 个用法
    MIDDLE( code: 1, description: "Middle"), 0 个用法
    SENIOR( code: 2, description: "Senior"), 0 个用法
    UNKNOWN( code: 3, description: "Unknown"); 1 个用法

    private final int code; 3 个用法
    private final String description; 3 个用法

    SkillLevel(int code, String description) { 8 个用法  @dingyan <dy_100000@163.com>
        this.code = code;
        this.description = description;
    }

    public int getCode() { return code; }

    public String getDescription() { return description; }

    public static List<String> getDescriptions() 1 个用法  @dingyan <dy_100000@163.com>
    {
        List<String> descriptions = new ArrayList<>();

        for (SkillLevel item : SkillLevel.values()) {
            descriptions.add(item.description);
        }

        return descriptions;
    }
}
```

- 定义OPC UA枚举类型

```
public class SkillLevelEnumType extends UaEnumDataType { 5 个用法
    public SkillLevelEnumType(EmployeeDataManager nodeManager)
    {
        super(
            dataTypeId: "SkillLevel",
            new LocalizedText("SkillLevel"),
            SkillLevel.getDescriptions(),
            nodeManager);
    }
}
```

— 课程1-节点管理器 (NodeManagerWebService)

- EmployeeDataManager (继承自 NodeManagerWebService)，负责管理模型，并接受来自UaServer对模型的请求
- 需重载onStartup，对EmployeeDataManager进行初始化，如创建模型，设置入口节点
- 通过addObjectType等接口将模型添加到NodeManager中
- 通过addRootObject设置入口数据节点

```
public class EmployeeDataManager extends NodeManagerWebService { 11 个用法  @dingyan <dy_100000@163.com>
    // Skill definitions
    public SkillLevelEnumType skillLevelEnumType; 3 个用法
    public SkillCategoryEnumType skillCategoryEnumType; 3 个用法
    public SkillType skillType; 3 个用法
    public SkillDirectoryType skillDirectoryType; 4 个用法

    > public EmployeeDataManager() { super(namespaceUri: "EmployeeDataManager"); }

    @Override 0 个用法  @dingyan <dy_100000@163.com>
    ~ public void onStartup() throws UaRuntimeException {
        // Create all types
        skillLevelEnumType = new SkillLevelEnumType(nodeManager: this);
        addDataType(skillLevelEnumType);

        skillCategoryEnumType = new SkillCategoryEnumType(nodeManager: this);
        addDataType(skillCategoryEnumType);

        skillType = new SkillType(skillLevelEnumType, skillCategoryEnumType, nodeManager: this);
        addObjectType(skillType);

        skillDirectoryType = new SkillDirectoryType(nodeManager: this);
        addObjectType(skillDirectoryType);

        addRootObject(skillDirectoryType.RootSkillDirectoryId, skillDirectoryType);
    }
}
```

— 课程1-OPC UA WebServer (UaWebServer)

- UaWebServer负责提供OPC UA Http服务
- 通过重载onStartUp对UA服务进行初始化
- 需将EmployeeDataManager注册到UaWebServer中，确保提供相应的信息模型
- 可通过设置UaServerConfigure对UA Server的服务参数进行设置
- 可通过resource中的application.properties中设置服务的参数，如URI，端口号，数据库连接等

```
@Service 0 个用法  @dingyan <dy_100000@163.com>
public class UaWebServer extends org.opcfoundation.webserver.service.UaWebServer {
    public UaWebServer() { super(); }

    @Override 0 个用法  @dingyan <dy_100000@163.com>
    public void onStartUp() {
        System.out.println("Start up server with NodeManager ");

        // Initialize OPC UA Web Server configuration
        configureServer();

        // Add opc ua data manager
        addNodeManager(new EmployeeDataManager());
    }

    @Override 0 个用法  @dingyan <dy_100000@163.com>
    public void onShutDown() { System.out.println("Shut down server with NodeManager"); }

    private void configureServer() 1 个用法  @dingyan <dy_100000@163.com>
    {
        UaServerConfigure configure = new UaServerConfigure();
        configure.setApplicationUri("EmployeeDemo");
        configure.setApplicationName(LocalizedText.english(text: "EmployeeDemo"));
        configure.setProductUri("EmployeeDemo");
        setServerConfigure(configure);
    }
}
```

— 课程1-总结

1. 目录对象：负责管理对象列表，需继承UaObjectDirectoryType，并重载getChildren返回对象列表及各对象的基本信息
2. 数据对象：负责管理单个对象的信息和服务，需继承UaDataObjectType，并重载getObjectAttribute获取对象的描述信息，并通过getVariableValues获取对象的数据
3. 枚举类型：负责定义枚举类型，可继承UaEnumDataType，并赋予数值相应的含义(如0-初级，1-中级，2-高级)
4. 模型管理：NodeManagerWebService负责对某一领域的模型定义进行综合管理
5. UA Web服务：负责提供OPC UA WebService，并将各NodeManagerWebService中的数据与服务提供给UA WebClient
6. 微服务：OPC UA WebService基于Spring框架进行扩展，因此从系统架构（微服务）到配置文件等方面，都完全继承自Spring框架，例如采用application.properties进行服务配置，可采用诸如@Server，@Component等注释

— 课程2

部门列表



项目列表



项目数据

Name	Data Type	Value
StartTime	DateTime	2024/5/1 00:00:00
EndTime	DateTime	2027/6/30 18:00:00
Budget	Float	10

ID	Name	Comment	ParentDepartmentId
1	研发中心	New tech R&D, including DCS/PLC/Industry soft...	NULL
2	平台所	Software and platform development	1
3	控制所	PLC programing tool and runtime development	1
4	通讯所	Industry protocol R&D	1
NULL	NULL	NULL	NULL

ID	Name	MissionDescription	StartTime	EndTime	Budget	DepartmentId
1	开放PLC	New generation controller platform	2024-01-01 00:00:00	2026-12-31 18:00:00	15	3
2	数字孪生	Digital twin platform based on information model	2024-05-01 00:00:00	2027-06-30 18:00:00	10	2
3	WebHMI	WebBased HMI	2024-03-01 00:00:00	2025-09-30 18:00:00	5	2
4	Profinet协议	Supporting Profinet in PLC	2024-06-01 00:00:00	2025-09-30 18:00:00	3	4

— 课程2-DepartmentType

类型声明

- ▼ BaseObjectType
 - ▶ FolderType
 - ▶ ServerType
 - ▶ ModellingRuleType
 - ▶ BaseEventType
 - 项目类定义
 - ▼ 部门类定义
 - ▶ 部门
 - ▶ 项目

对象实例

- ▼ Departments
 - ▼ 研发中心
 - ▼ 部门
 - ▶ 平台所
 - ▶ 控制所
 - ▶ 通讯所
 - ▶ 项目

- DepartmentType需同时提供下级部门列表
DepartmentDirectoryType
和项目列表
ProjectDirectoryType
两个子模型(Sub-model)
- 由于由多个子模型组合, 因此
DepartmentType这类模型又被称为主模型(Master-Model)

- ▼ 部门类定义
 - ▶ 部门
 - ▶ 项目

- DepartmentType需继承
UaMasterObjectType, 从而具备提供
子模型的能力

```
public class DepartmentType extends UaMasterObjectType { 4个用法

    public DepartmentType( 1个用法  @dingyan <dy_100000@163.com>
        DepartmentDirectoryType departmentDirectoryType,
        ProjectDirectoryType projectDirectoryType,
        EmployeeDataManager nodeManager)
    {
        super( objectId: "DepartmentType", new LocalizedText("DepartmentType"))
    }
}
```

- DepartmentType需通过addSubmodel,
添加DepartmentDirectoryType和
ProjectDirectoryType两个子模型

```
// Add department directory to get child department
addSubmodel(
    childId: "Departments",
    mandatory: true,
    new LocalizedText("部门"),
    new LocalizedText("归属该部门管理的下级部门"),
    departmentDirectoryType);

// Add project directory to get project of department
addSubmodel(
    childId: "Projects",
    mandatory: true,
    new LocalizedText("项目"),
    new LocalizedText("归属该部门的项目"),
    projectDirectoryType);
```


— 课程2-DepartmentDirectoryType

类型声明

- ▼ **T** BaseObjectType
 - ▼ **T** FolderType
 - T** 项目目录类定义
 - ▶ **T** 部门目录类定义

对象实例

- ▼ **Departments**
 - ▶ **研发中心**
- ▼ **研发中心**
 - ▼ **部门**
 - ▶ **平台所**
 - ▶ **控制所**
 - ▶ **通讯所**

- DepartmentDirectoryType负责展示所有一级部门，或某一部门的下级部门

- 当作为一级部门列表时，需要调用addRootObject添加到根目录

```
addRootObject(departmentDirectoryType.RootDepartmentDirectoryId, departmentDirectoryType);
```

- 当作为某一部门的下级部门时，需调用addSubmodel将其添加到DepartmentType中

```
// Add department directory to get child department  
addSubmodel(  
    childId: "Departments",  
    mandatory: true,  
    new LocalizedText("部门"),  
    new LocalizedText("归属该部门管理的下级部门"),  
    departmentDirectoryType);
```

- 在getChildren中，需要根据request中的ID来判定，需要返回的是所有一级部门信息，或是某一部门的下级部门信息

- 当ID为RootDepartmentDirectoryId时，需查询所有一级部门信息，否则将查询某一部门的下级部门

```
if (request.getObjectId().getId().equals(RootDepartmentDirectoryId))  
{  
    // If is root department, parent id is null  
    parentDepartmentId = null;  
} else {  
    // Provide parent department id  
    parentDepartmentId = Integer.parseInt(request.getObjectId().getId());  
}
```

```
<select id="getChildDepartment" resultMap="DepartmentResult">  
    SELECT ID, Name, Comment, ParentDepartmentId FROM Department WHERE  
    <if test="id != null">  
        ParentDepartmentId = #{id}  
    </if>  
    <if test="id == null">  
        ParentDepartmentId IS NULL  
    </if>  
</select>
```

— 课程2-ProjectType/ProjectDirectoryType

对象实例



对象数据

Name	Data Type	Value
StartTime	DateTime	2024/5/1 00:00:00
EndTime	DateTime	2027/6/30 18:00:00
Budget	Float	10

- ProjectDirectoryType用于展示部门下管理的研发项目列表，因此需继承UaObjectDirectoryType
- 在getChildrent中，需要通过DepartmentId查询并返回Project信息

```
public class ProjectDirectoryType extends UaObjectDirectoryType {  
    public ProjectDirectoryType(EmployeeDataManager nodeManager) {  
        super( objectTypeId: "ProjectDirectoryType", new LocalizedText
```

```
// Get department id from request  
Integer departmentId = Integer.parseInt(request.getObjectId().getId());  
  
// Database query  
ProjectMapper projectMapper = EmployeeDatabase.getProjectMapper();  
List<Project> projects = projectMapper.getProjectOfDepartment(departmentId);  
  
// Return general information of project  
List<UaChildObjectDescriptor> children = new ArrayList<>();  
ProjectType projectType = ((EmployeeDataManager)nodeManager).projectType;
```

- ProjectType用于展示每个项目的信息，因此继承自UaDataObjectType，并通过getObjectAttribute和getVariableValues获取信息和数据

```
public class ProjectType extends UaDataObjectType {  
    private final UaVariable startTime; 2个用法  
    private final UaVariable endTime; 2个用法  
    private final UaVariable budget; 2个用法
```

```
// Get project id from request  
Integer projectId = Integer.parseInt(request.getObjectId().getId());  
  
// Database query  
ProjectMapper projectMapper = EmployeeDatabase.getProjectMapper();  
Project project = projectMapper.getProjectInfo(projectId);  
if (null == project) throw new UaRuntimeException(StatusCodes.Bad_NodeIdUnknown);  
  
// Return general information  
ReadObjectAttributeResponse response = new ReadObjectAttributeResponse(  
    new LocalizedText(project.getName()),  
    new LocalizedText(project.getDescription()));
```

```
Integer projectId = Integer.parseInt(request.getObjectId().getId());  
  
// Database query  
ProjectMapper projectMapper = EmployeeDatabase.getProjectMapper();  
Project project = projectMapper.getProjectData(projectId);  
if (null == project) throw new UaRuntimeException(StatusCodes.Bad_NodeIdUnknown);  
  
// Return field data  
ReadVariableValueResponse response = new ReadVariableValueResponse();  
ZoneId zoneId = ZoneId.systemDefault();  
for (UaChildId item: request.getVariableIds())  
{  
    // Return start time if provided
```

— 课程2-总结

1. 主模型：如果一个模型是由多个片段的数据和功能组成的（如部门需要由下级部门和项目列表组成），则需要将此模型认定为主模型，并通过添加子模型
2. 子模型：可被挂在主模型下的模型，其中UaDataObjectType, 用于描述主模型某一片段的数据，而UaObjectDirectoryType，则负责管理主模型下的子项
3. 子模型作用域：当子模型被挂在到主模型上时，其数据与功能则会随着主模型的作用域走，例如DepartmentDirectoryType，该模型在某个部门下时，只负责展示该部门的下级部门；而当将其挂载到入口节点时，则负责展示所以一级部门的信息

— 课程3

部门列表

- ▾ Departments
 - ▾ 研发中心
 - ▾ 部门
 - 平台所
 - 控制所
 - 通讯所
 - 员工

EmployeeId	DepartmentId
1	2
2	2
3	3
4	3
5	3

人员列表

- ▾ 平台所
 - 部门
 - ▾ 员工
 - Zhang
 - Li

	ID	Name
▶	1	Zhang
	2	Li
	3	Wang
	4	Zhao
	5	Sun
⊛	NULL	NULL

技能列表

- ▾ Li
 - 基本信息
 - ▾ 技能
 - BE-DEV-S
 - UI-DEV-M
 - 能力

ID	Sex	Birthday	PhoneNumber	Address	EmployeeId
1	1	1990-06-01 00:00:00	XXXXXXXXXXXX	XX,Beijing	1
2	0	1993-07-12 00:00:00	XXXXXXXXXXXX	XX,Shanghai	2
3	1	1987-04-23 00:00:00	XXXXXXXXXXXX	XX,Hangzhou	3
4	0	1999-01-25 00:00:00	XXXXXXXXXXXX	XX,HaErBin	4
5	1	1997-12-19 00:00:00	XXXXXXXXXXXX	XX,Tianjin	5
NULL	NULL	NULL	NULL	NULL	NULL

能力评估服务

- ▾ Li
 - 基本信息
 - 技能
 - ▾ 能力
 - 全栈开发者
 - 资深测试

Call Method

Input Arguments

Name	Value	Data Type	Description
IsCompetent	true	Boolean	IsCompetent

Output Arguments

Name	Value	Data Type	Description
IsCompetent	true	Boolean	IsCompetent

Result

Succeeded

Close Call

EmployeeId	SkillId
1	1
2	3
2	5
3	3
4	2
4	6
4	7
5	8

— 课程3-EmployeeType

类型声明

- ▼  员工类定义
 - ▶  基本信息
 - ▶  技能
 - ▶  能力

对象实例

- ▶  Zhang
- ▼  Li
 - ▶  基本信息
 - ▶  技能
 - ▶  能力

- EmployeeType由以下三个子模型构成
基本信息-EmployeeInfoType
技能列表-SkillDirectoryType
能力评估-EmployeeCompetenceType
- EmployeeType需要继承自
UaMasterObjectType, 并通过
addSubmodel添加以下三个类型

```
public class EmployeeType extends UaMasterObjectType {  
    public EmployeeType( 1 个用法  @dingyan <dy_100000@10  
        EmployeeInfoType employeeInfoType,  
        SkillDirectoryType skillDirectoryType,  
        EmployeeCompetenceType employeeCompetenceTy  
        EmployeeDataManager nodeManager)  
    {  
        super( objectId: "EmployeeType", new Localized  
  
        // Add employee o to get child department  
        addSubmodel(  
            childId: "Information",  
            mandatory: true,  
            new LocalizedText("基本信息"),  
            new LocalizedText("提供员工的基本信息"),  
            employeeInfoType);  
    }
```

— 课程3-EmployeeInfoType/SkillDirectoryType

类型声明

▸ T 员工信息类定义

对象实例

▼ Li

▸ 基本信息

对象数据

Name	Data Type	Value
Sex	Sex	0(Male)
Birthday	DateTime	1993/7/12 00:00:00
PhoneNumber	String	XXXXXXXXXXXX
Address	String	XX, Shanghai

- EmployeeInfoType继承自UaDataObjectType, 并通过getVariableValues返回员工的信息

```
public class EmployeeInfoType extends UaDataObjectType {  
    private final UaVariable sex; 2个用法  
    private final UaVariable birthday; 2个用法  
    private final UaVariable phoneNumber; 2个用法  
    private final UaVariable address; 2个用法  
  
    public EmployeeInfoType( 1个用法 @dingyan <dy_1000000  
        SexEnumType sexEnumType,  
        EmployeeDataManager nodeManager) {  
        super( objectId: "EmployeeInfoType", new Localiz
```

```
// Get employee id from request  
Integer employeeId = Integer.parseInt(request.getObjectId().getId());  
  
// Database query  
EmployeeMapper employeeMapper = EmployeeDatabase.getEmployeeMapper();  
EmployeeInfo employee = employeeMapper.getEmployeeInfo(employeeId);  
if (null == employee) throw new UaRuntimeException(StatusCodes.Bad_No  
  
// Return field data  
ReadVariableValueResponse response = new ReadVariableValueResponse();  
ZoneId zoneId = ZoneId.systemDefault();
```

- SkillDirectoryType作为EmployeeType的子模型时, 其作用为展现员工的技能, 因此需要在getChildren中, 根据EmployeeId查询

```
// Database query  
SkillMapper skillMapper = EmployeeDatabase.getSkillMapper();  
List<Skill> skills;  
  
if (request.getObjectId().getId().equals(RootSkillDirectoryId))  
{  
    // If is root skill directory, return all skills  
    skills = skillMapper.getAllSkillInfo();  
}  
else {  
    // Get employee id from request  
    Integer employeeID = Integer.parseInt(request.getObjectId().getId());  
    skills = skillMapper.getSkillInfoOfEmployee(employeeID);  
}
```

类型声明

▼ T BaseObjectType

▼ T FolderType

T 技能目录类定义

对象实例

▼ Li

▸ 基本信息

▼ 技能

▸ BE-DEV-S

▸ UI-DEV-M

— 课程3-EmployeeCompetenceType

类型声明

▼ 员工能力类定义

- ▶ 全栈开发者
- ▶ 资深测试

对象实例

▼ Li

- ▶ 基本信息
- ▶ 技能
- ▼ 能力
 - ▶ 全栈开发者
 - ▶ 资深测试

方法调用

Call Method

Input Arguments			
Name	Value	Data Type	Description
Output Arguments			
Name	Value	Data Type	Description
IsCompetent	True	Boolean	IsCompetent

- EmployeeCompetenceType用于根据员工的Skill(技能)，对员工能力进行定性评估（如全栈开发，资深测试）
- 这些评估不来自于数据库，而是经过算法计算返回
- 针对此类需要借助算法，进行定性评估的功能，可采用方法调用来表达

▼ 能力

- ▶ 全栈开发者
- ▶ 资深测试

- 在EmployeeCompetenceType中，调用addMethod来添加方法

```
fullStackDeveloper = addMethod(  
    memberId: "FullStackDeveloper",  
    mandatory: true,  
    new LocalizedText("全栈开发者"),  
    new LocalizedText("If this pers  
    inputArguments: null,  
    outputArguments);
```

- 重载methodCall，编写方法调用实现

```
public CompletableFuture<MethodCallResponse> methodCall(MethodCallRequest request)  
try {  
    // Get employee id from request  
    Integer employeeId = Integer.parseInt(request.getObjectId().getId());  
    List<Variant> outputArguments = new ArrayList<>();  
  
    // Database query  
    SkillMapper skillMapper = EmployeeDatabase.getSkillMapper();  
    List<Skill> skills = skillMapper.getSkillDataOfEmployee(employeeId);  
  
    if (request.getMethodName().equals(fullStackDeveloper.browseName()))  
    {  
        Boolean result = isFullStackDeveloper(skills);  
        outputArguments.add(Variant.ofBoolean(result));  
        return CompletableFuture.completedFuture(new MethodCallResponse(output
```

— 课程3-总结

1. 对象逻辑：如果需要对模型提供动态的逻辑处理能力，可通过在模型上定义方法(Method)，并设置方法的输入输出参数；
2. 方法调用：当需要触发这个方法时，OPC UA Client会调用Call接口，在Call时需提供入参(InputArguments)，执行结束后，服务端会返回出参(OutputArguments)
3. 方法执行：当UA Server接收到Call调用时，会触发模型的methodCall，并执行开发者编写的代码逻辑



谢谢!

Intelligence For Excellence